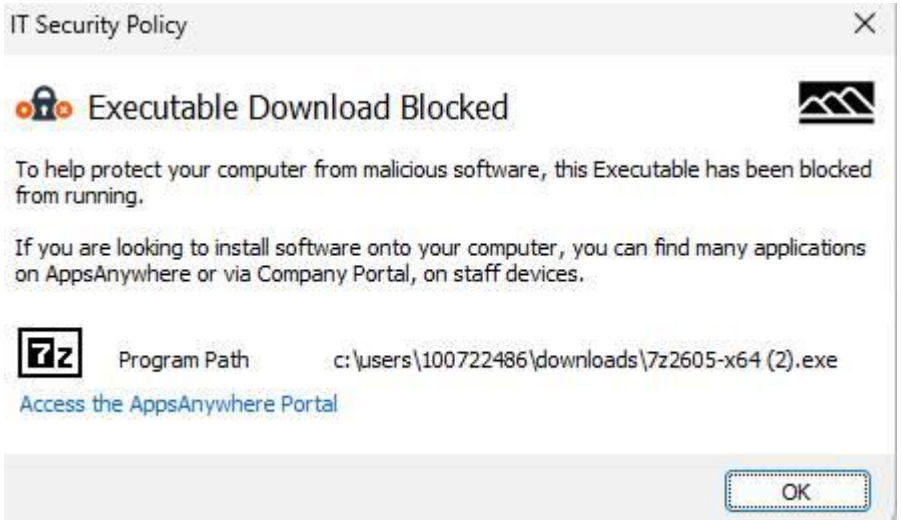
Week 1 Submission – Network Walks internship work – Arnav Bhagat

Download 7-Zip 26.05 (2026-10-21) for Windows:

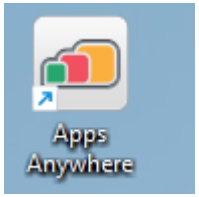
Link	Type	System	Description
Download	.exe	64-bit Windows x64	7-Zip installer for Windows
Download	.exe	32-bit Windows x86	
Download	.exe	64-bit Windows arm64	
Download	.msi	64-bit Windows x64	(alternative MSI installer) 7-Zip for 64-bit Windows x64
Download	.msi	32-bit Windows x86	(alternative MSI installer) 7-Zip for 32-bit Windows
Download	.7z	Windows x86 / x64	7-Zip Extra: standalone console version, 7z DLL, Plugin for Far Manager
Download	.tar.xz	64-bit Linux x86-64	7-Zip for Linux: console version
Download	.tar.xz	32-bit Linux x86	
Download	.tar.xz	64-bit Linux arm64	
Download	.tar.xz	32-bit Linux arm	
Download	.tar.xz	macOS (arm64 / x86-64)	7-Zip for MacOS: console version
Download	.7z	any / Windows	7-Zip Source code
Download	.tar.xz	any / Windows	7-Zip Source code
Download	.7z	any / Windows	LZMA SDK: (C, C++, C#, Java)
Download	.exe	Windows	7zr.exe (x86) : 7-Zip console executable

We recommend to use **exe** type installer instead of **msi** installer version.

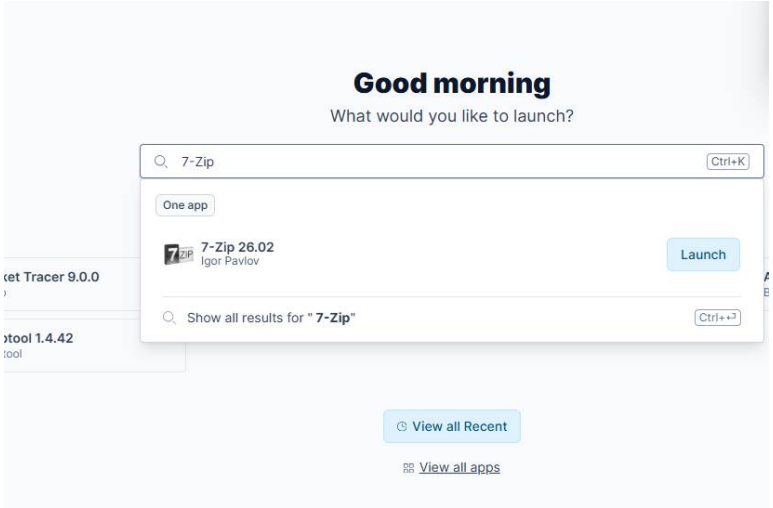
The initial phase of the Week 1 Network Walls infrastructure deployment required downloading a local archive extraction utility to process future laboratory compressed images. Following the architecture specifications, the 7-Zip official download matrix was accessed. The 64-bit Windows x64 .exe installer was selected, directly aligning with the host hardware architecture and adhering to the platform's vendor documentation which prioritizes executable binary installs over .msi packages for client devices.



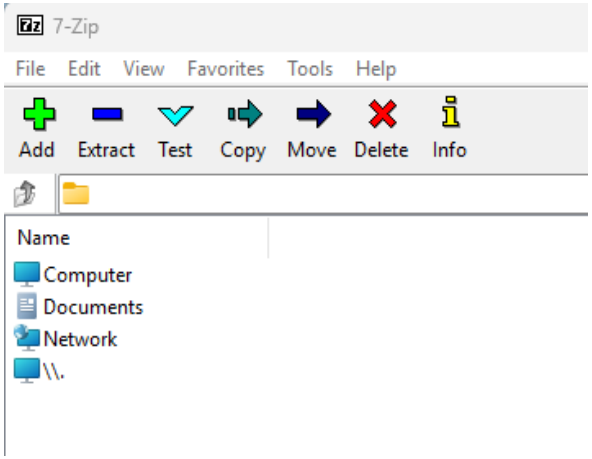
Upon attempting to run the downloaded 7z2605-x64.exe installer binary, the managed host endpoint triggered an institutional University Of Derby IT Security Policy enforcement block. The operating system intercepted the execution layer, displaying an *Executable Download Blocked* restriction dialogue box. This defensive measure explicitly prevents non-whitelisted binaries from modifying system folders or memory spaces on corporate/academic devices, directing the user instead to utilize the authorized AppsAnywhere Portal or deployment store to acquire pre-vetted corporate tools.



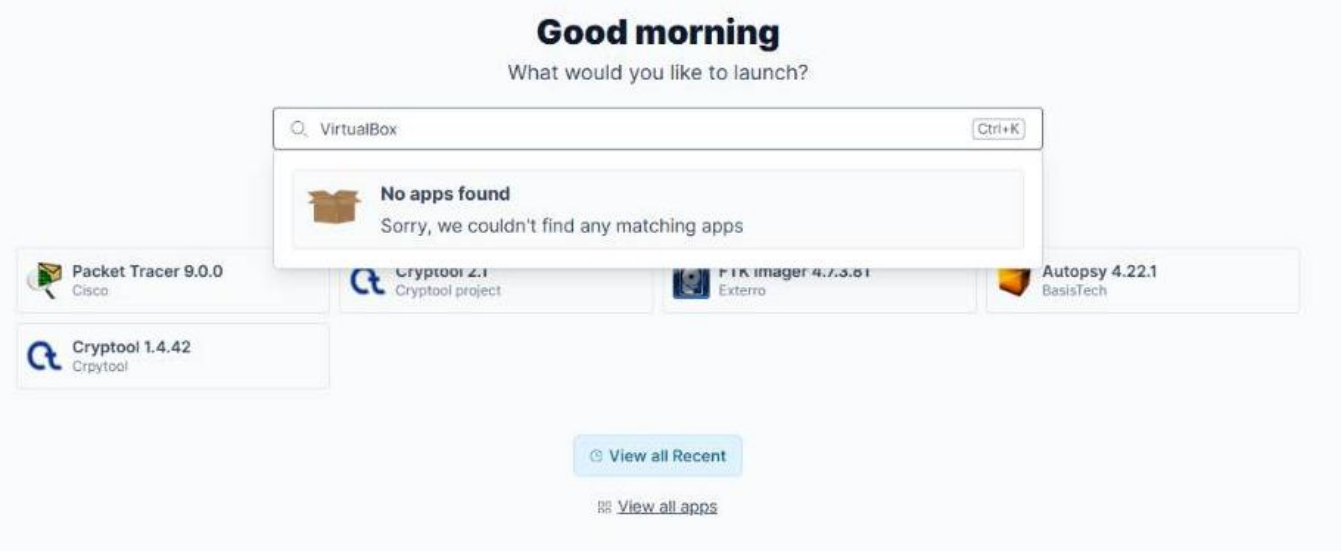
To pivot away from the blocked web installer, the institutional AppsAnywhere client software shortcut was launched from the host desktop. Utilizing this endpoint wrapper allows the system to interact securely with the organization's pre-approved application streaming environment, enabling software deployment that complies fully with the active security profile.



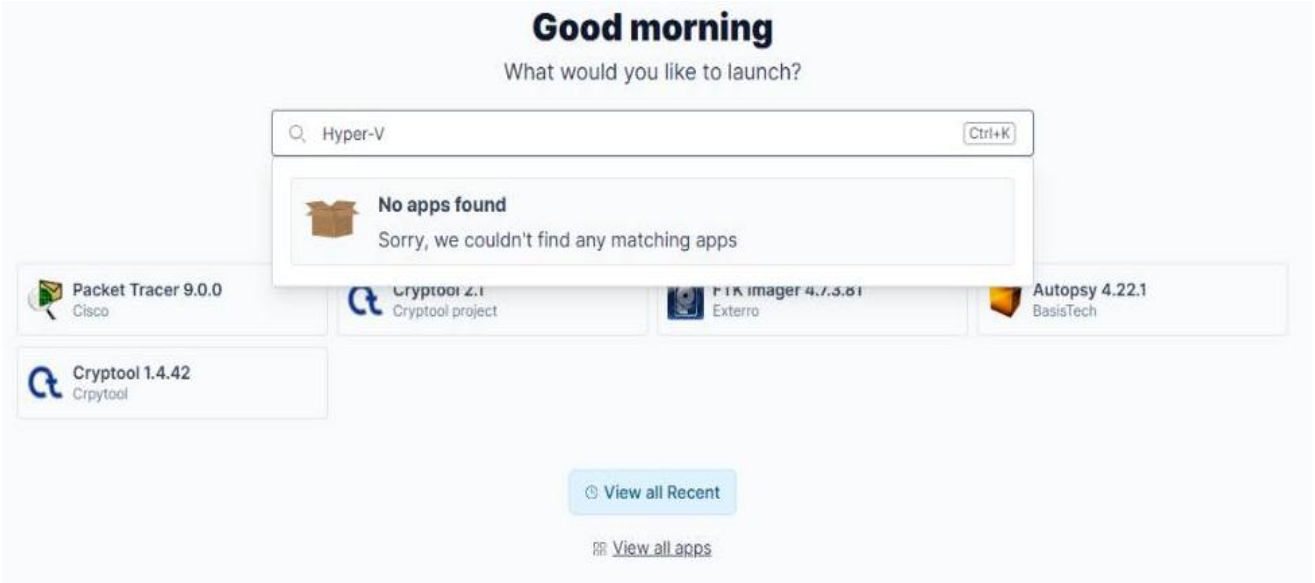
Inside the AppsAnywhere web portal storefront, a search query was executed for the utility application. 7-Zip 26.02 was successfully located within the institutional repository as a verified asset. Clicking the blue Launch option initiated the secure Cloudpaging deployment mechanism, dynamically pulling down the virtualized package to the workstation without requiring standard local administrator execution privileges.



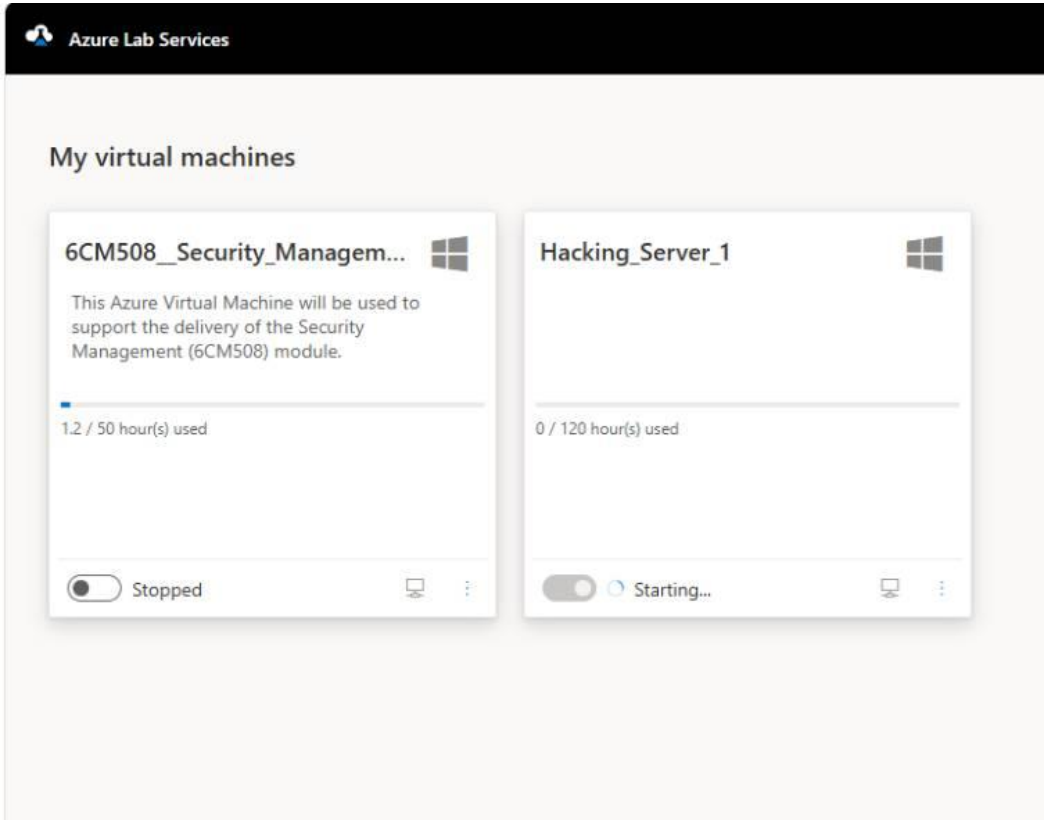
The streaming provisioning cycle finalized successfully, culminating in the initialization of the 7-Zip File Manager interface window on the workstation. The active dashboard verifies that the archiving engine is fully operational and recognized by the system workspace, successfully completing Phase 1 of the lab framework despite local policy constraints.



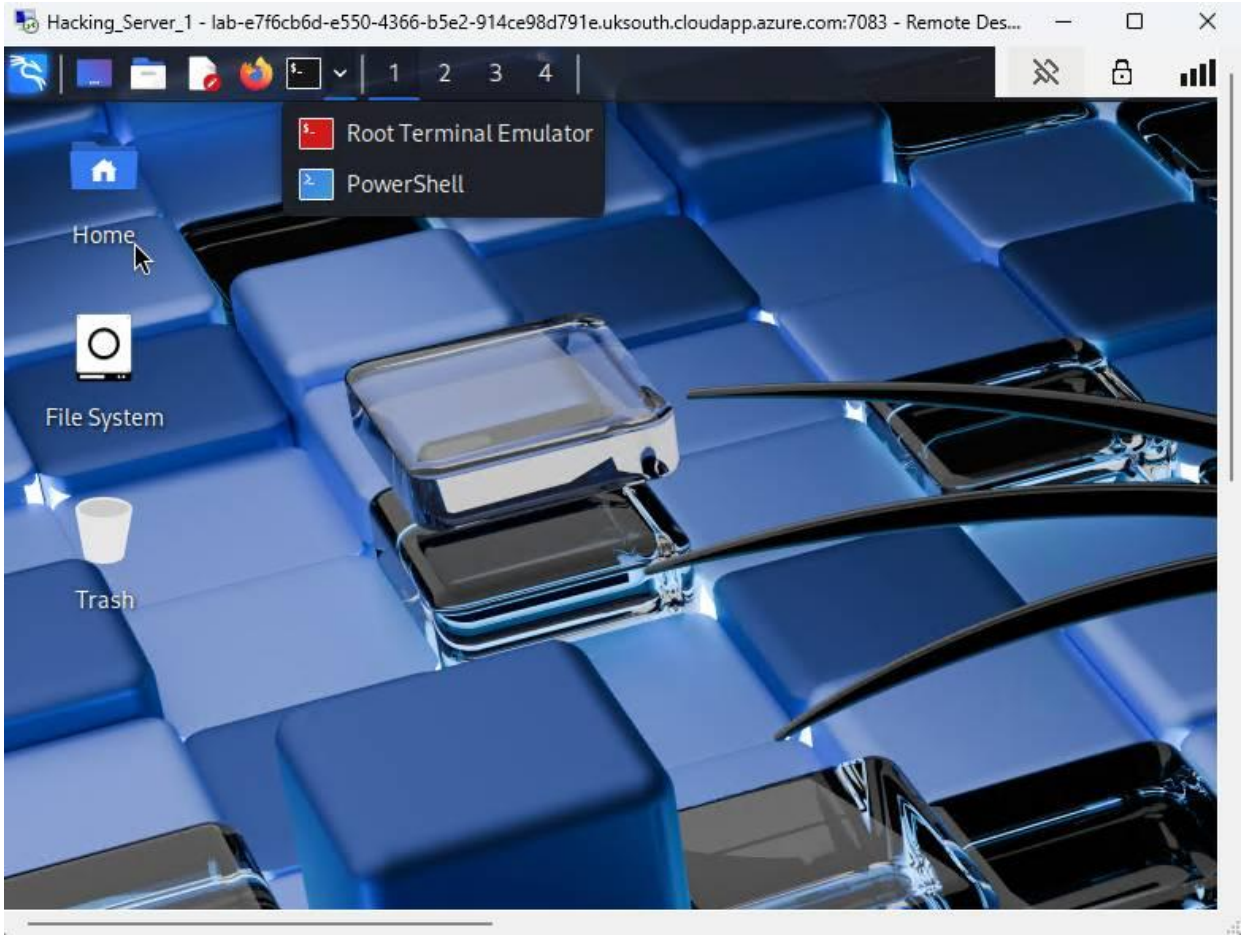
Following the initial laboratory guide instructions, a search query was executed within the AppsAnywhere repository for Oracle VirtualBox. The portal returned a *"No apps found"* status, confirming that a standard type-2 hypervisor engine was not provisioned or whitelisted for this specific student host profile. This limitation blocked the standard route of building local host-only virtual machines.



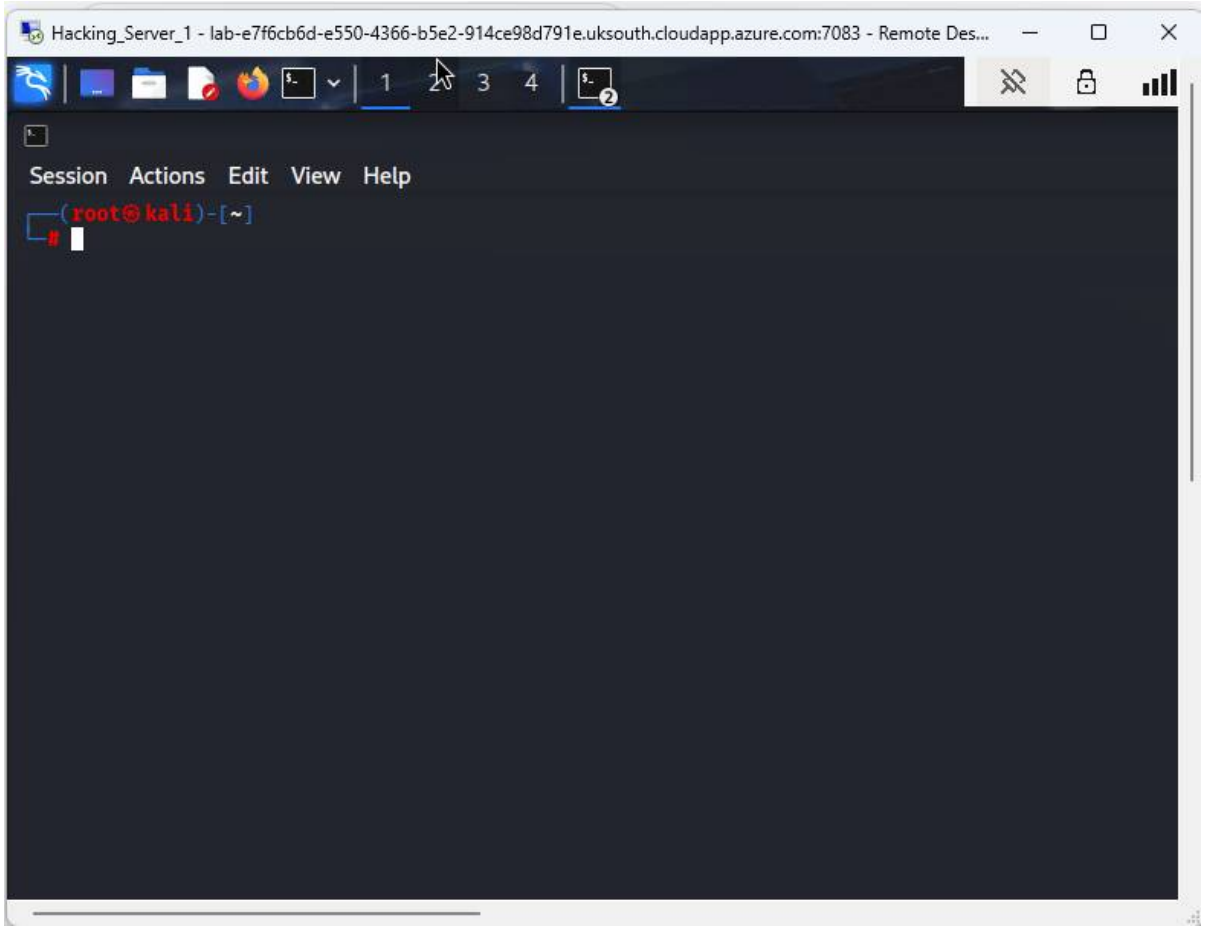
To find a workaround for the missing software library assets, a secondary search was performed for native platform alternatives, specifically targeting Microsoft Hyper-V. The catalog confirmed that no secondary hardware virtualization wrappers were available for dynamic streaming deployment either. This confirmed that the local environment lacked the software required to manage virtualized networking subnets natively, necessitating a pivot to external infrastructure.



Due to the software installation constraints and hypervisor deficiencies on the local workstation, the laboratory workflow was successfully migrated to the cloud using my University 'Azure Lab Services portal'. Two virtual assets assigned to the academic module are present: a security management workstation and Hacking_Server_1. The toggle switch for Hacking_Server_1 was activated, moving its status to *"Starting..."*. Deploying this cloud-hosted environment completely bypasses the local endpoint security blocks and provides a fully isolated cloud sandbox suitable for network topology manipulation.



After establishing a secure Remote Desktop Protocol (RDP) handshake with the Hacking_Server_1 cluster in the cloud, the desktop workspace of the Kali Linux distribution initialized successfully. To begin configuring the static IP infrastructure required by Step 5 of the laboratory guide, administrative network privileges were required. The applications menu dropdown was selected, highlighting the choice between PowerShell and the native Root Terminal Emulator. The Root Terminal option was selected to launch an elevated shell instance with full superuser (root) access, ensuring subsequent interface binding commands run with the necessary security permissions.



The Root Terminal Emulator opened successfully, establishing an interactive shell session running with elevated privileges as indicated by the default prompt string root@kali. Operating inside this superuser context is a crucial prerequisite for executing low-level networking reconfigurations. This blank terminal window serves as the administrative launchpad for executing NetworkManager tools (nmcli) to manually decouple the system from dynamic address leasing (DHCP) and bind the static lab parameters.

```
┌──(root㉿kali)-[~]
└─# nmcli connection show
NAME                UUID                                  TYPE      DEVICE
Wired connection 1  da54f927-f9f7-3345-832a-9fa5d2df7b6a  ethernet  eth0
lo                  6b67ad44-e87c-4689-a319-9a057cc56514  loopback  lo

┌──(root㉿kali)-[~]
└─# nmcli connection modify "Wired connection 1" ipv4.method manual
Error: Failed to modify connection 'Wired connection 1': ipv4.method: method 'manual' requires at least an address or a route

┌──(root㉿kali)-[~]
└─# nmcli connection show
NAME                UUID                                  TYPE      DEVICE
Wired connection 1  da54f927-f9f7-3345-832a-9fa5d2df7b6a  ethernet  eth0
lo                  6b67ad44-e87c-4689-a319-9a057cc56514  loopback  lo

┌──(root㉿kali)-[~]
└─# nmcli connection modify "Wired connection 1" ipv4.addresses 10.0.0.2/24 ipv4.method manual

┌──(root㉿kali)-[~]
└─# nmcli connection modify "Wired connection 1" ipv4.gateway 10.0.0.1

┌──(root㉿kali)-[~]
└─# nmcli connection up "Wired connection 1"
Connection successfully activated (D-Bus active path: /org/freedesktop/NetworkManager/ActiveConnection/3)
```

- The networking reconfiguration was systematically processed using the NetworkManager Command Line Interface (nmcli).
    1. Profile Identification: Running nmcli connection show revealed the primary network profile label was "Wired connection 1", bound to physical adapter device eth0.
    2. Error Diagnosis (The Catch-22): An initial attempt to independently switch the connection mode via nmcli connection modify "Wired connection 1" ipv4.method manual was rejected by the system. The platform generated a critical error stating that manual assignment requires an accompanying static ip definition or routing rule.
    3. Parameter Correction: To satisfy the NetworkManager syntax parameters, the properties were combined into a singular line transaction: nmcli connection modify "Wired connection 1" ipv4.addresses 10.0.0.2/24 ipv4.method manual. This declared the needed static endpoint address alongside the methodology parameter cleanly.
    4. Gateway Mapping: The gateway route was designated by assigning ipv4.gateway 10.0.0.1.
    5. Interface Reload and Activation: The configuration change was committed across the subsystem using the command nmcli connection up "Wired connection 1". The execution returned a positive response confirming that the network connection profile was successfully activated via the active D-Bus platform route.



To verify the new settings on the network interface card, the command ip addr show eth0 was executed in the console. The terminal output confirms that the network subsystem successfully updated the interface state:

1. Static IP binding: The primary interface adapter configuration displays inet 10.0.0.2/24, proving that the static IP address has been successfully bound to device eth0.
2. Broadcast address mapping: The interface has correctly configured its local broadcast layer to brd 10.0.0.255, ensuring standard packet distribution behavior across the new /24 subnet.
3. Interface state: The operational flags display <BROADCAST,MULTICAST,UP,LOWER_UP>, confirming that the underlying network adapter is online, active, and fully ready to communicate.

With the manual addressing parameters confirmed, the Kali Linux environment is properly isolated on the target subnet, successfully concluding the milestones required for Week 1.